Technische Hochschule Würzburg-Schweinfurt
Fakultät Informatik und Wirtschaftsinformatik

Bachelorarbeit

Comparative Analysis of Model and Feature Complexity in Financial Time Series Forecasting

vorgelegt an der Technischen Hochschule Würzburg-Schweinfurt in der Fakultät Informatik und Wirtschaftsinformatik zum Abschluss eines Studiums im Studiengang Informatik

N. N.

Eingereicht am: April 8, 2026

Erstprüfer: Prof. Andreas Lehrmann
Zweitprüfer: Prof. Pascal Meißner



Zusammenfassung

Ziel dieser Arbeit ist es, den Zusammenhang zwischen aufbereiteten Features und Modellarchitekturen zu evaluieren. Die Forschungsfrage „Comparative Analysis of Model and Feature Complexity in Financial Time Series Forecasting“ untersucht, ob mehr Aufwand in die Feature-Generierung aufgewendet werden sollte oder ob komplexere Modelle trotz des damit verbundenen Rechenaufwands bessere Ergebnisse erzielen.

Um diese Frage zu beantworten, wurde die Zusammensetzung des Nasdaq-100-Index herangezogen. Der genutzte Datensatz umfasst 101 Börsenticker über den Zeitraum von 2015 bis 2023 in einem 5-Minuten-Intervall. Die daraus abgeleiteten Features wurden in drei Komplexitätsstufen unterteilt, woraus sieben verschiedene Permutationen entstehen. Jede ausgewählte Modellarchitektur wurde mit jeder dieser Permutationen trainiert. Zu diesem Zweck wurde eine Pipeline für maschinelles Lernen in Python implementiert, welche eine detaillierte Auswertung statistischer Metriken und ökonomischer Resultate ermöglicht.

Der analytische Vergleich zeigt, dass über alle Modellarchitekturen und Feature-Komplexitätsstufen hinweg keine signifikante Verbesserung erzielt wurde. Die Modelle konvergierten konsistent zu einem Mean-Predictor, welcher lediglich den Erwartungswert der Zielvariable vorhersagt. Selbst Modelle auf dem aktuellen Stand der Technik konnten in Verbindung mit komplexen Features keine Verbesserung erzielen.

Diese Ergebnisse unterstreichen die Schwierigkeit von Intraday-Finanzprognosen. Finanzdaten weisen Eigenschaften wie Nichtstationarität und ein geringes Signal-Rausch-Verhältnis auf, was es extrem schwer macht, prägnante Signale zu extrahieren. Die Resultate stehen damit im Einklang mit der Markteffizienzhypothese (EMH), welche besagt, dass Marktpreise bereits alle verfügbaren Informationen widerspiegeln und eine Vorhersage von Preisbewegungen auf Basis historischer Daten kaum möglich ist.

Während die Literatur für tägliche Renditen oft profitable Szenarien beschreibt, zeigt diese Arbeit eine Komplexitätsgrenze für technische Indikatoren im Intraday-Bereich auf. Zukünftige Forschung könnte davon profitieren, alternative Datenquellen wie Sentimentanalysen oder Limit-Order-Book-Daten mit einzubeziehen, um einen Informationsvorteil zu erlangen, den rein preisbasierte Features nicht bieten können.

Abstract

The goal of this thesis is to evaluate the relationship between engineered features and model architectures. Therefore, the research question "Comparative Analysis of Model and Feature Complexity in Financial Time Series Forecasting" investigates whether more effort should be put into feature engineering or whether more complex models should be trained despite the associated computational effort.

To address this question, the composition of the Nasdaq-100 index was utilized. The used dataset comprises data from 101 tickers over the period from 2015 to 2023 at a 5-minute interval. Derived features were categorized into three hierarchical complexity levels, resulting in seven distinct permutations. Each selected model architecture was trained on each permutation. For this purpose, a robust machine learning pipeline was implemented in Python to enable systematic training and evaluation using machine learning metrics and economic evaluations.

The comparative analysis highlights that across all model architectures and feature complexity levels, no significant increase in performance was achieved. The models consistently converged toward mean-predictions, forecasting the expected value of the target distribution. Even state-of-the-art models with advanced feature sets could not achieve better results.

These results underscore the difficulty of forecasting financial data at an intraday frequency. Financial data exhibit non-stationarity and a low signal-to-noise ratio, making it extremely challenging to extract predictive signals. This aligns with the Efficient Market Hypothesis (EMH), which states that information is rapidly incorporated into prices.

While literature on daily returns often describes profitable scenarios, this thesis identifies a complexity limit for technical indicators in the intraday domain. Future research could benefit from utilizing alternative data, such as sentiment or limit order book data, that provide an informational advantage that purely price-derieved features lack.

Acknowledgments

I would like to thank my supervisor, Professor Lehrmann, for his time and advice. His insights into practical considerations and his experience in applying these methods in a real-world context were truly helpful for this thesis. I also want to thank Professor Meißner, in whose course the initial idea of practically applying machine learning methods first emerged.

Contents

List of Abbreviations viii

List of Figures x

List of Tables xi

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Efficiency vs. Predictability . . . 1
- 1.3 Goal of the Thesis . . . 2
- 1.4 Structure of the Thesis . . . 2

2 Literature Review 3

3 Technical Prerequisites 6
- 3.1 Definitions and Forecasting Strategies . . . 6
- 3.2 Forecasting in the Financial Domain . . . 7
- 3.3 Prediction Models . . . 8
  - 3.3.1 Linear Regression . . . 8
  - 3.3.2 Random Forest . . . 9
  - 3.3.3 Multilayer Perceptron . . . 9
  - 3.3.4 Transformer Architecture . . . 11

4 Methodology 13
- 4.1 Forecasting Strategy . . . 13
- 4.2 Global Model Approach . . . 13
- 4.3 Target Variable Calculation . . . 13
- 4.4 Feature-Complexity Concept . . . 14
- 4.5 Model-Complexity Concept . . . 15
- 4.6 Overview . . . 16

5 Implementation 17
- 5.1 Pipeline Definition . . . 17
- 5.2 Configuration-Driven Execution . . . 17
- 5.3 Pipeline Execution Workflow . . . 18

6 Experiments **20**
6.1 Data Analysis . . . 20
6.1.1 Data Selection . . . 20
6.1.2 Statistical Analysis . . . 20
6.1.3 Data Partitioning . . . 23
6.1.4 Preprocessing . . . 24
6.2 Baselines . . . 27
6.2.1 Overview of All features . . . 27
6.2.2 Models . . . 28
6.2.3 Model Categorization Matrix . . . 30
6.2.4 Complexity Hierarchy Justification . . . 30
6.3 Experimental Setup . . . 30
6.4 Metrics . . . 31
6.4.1 Statistical Metrics . . . 31
6.4.2 Economic Evaluation - Mean-Variance Analysis . . . 31
6.5 Training Process with Hyperparameter Tuning . . . 32
6.5.1 Final Model Parameters . . . 32
6.6 Quantitative Analysis . . . 33
6.6.1 Machine Learning Performance Metrics . . . 33
6.6.2 Economic Evaluation Metrics . . . 34
6.7 Qualitative Analysis . . . 39

7 Conclusion **42**
7.1 Summary of Findings . . . 42
7.2 Future Research . . . 42

Appendix **43**
Appendix A: Detailed Feature Specification . . . 43
Appendix B: Setup Configuration (config.json) . . . 49
Appendix C: Pipeline Class . . . 50
Appendix D: Base Model Interface . . . 50
Appendix E: Base Model Interface . . . 51
Appendix F: Feature Configuration (feature_config.json) . . . 51
Appendix G: RF Parameter Space . . . 53
Appendix H: MLP Parameter Space . . . 53
Appendix I: PatchTST Parameter Space . . . 54

Listings **57**

Literature **58**

Declaration on Oath **60**

Consent to Plagiarism Check **61**

List of Abbreviations

ARIMA AutoRegressive Integrated Moving Average. 4

CNN Convolutional Neural Network. 4

EMH Efficient Market Hypothesis. iv, 1, 39, 41

GNN Graph Neural Network. 4

IPO Initial Public Offering. 20

IQR Interquartile Range. 25, 26, 27

LLM Large Language Model. 3, 12

LSTM Long Short-Term Memory. 1, 3, 4

MAE Mean Absolute Error. 31, 32, 33, 35, 41, 42

MLP Multi-Layer Perceptron. xi, 3, 4, 9, 10, 29, 30, 33, 40, 54

MSE Mean Squared Error. 9, 31, 33, 35, 40, 42

MVO Mean-Variance Optimization. 31

OHLCV Open, High, Low, Close, Volume. 14, 17, 18, 20, 41, 42

OLS Ordinary Least Squares. 8, 28, 32

PatchTST Patch Time Series Transformer. xi, 3, 4, 29, 30, 33, 40, 41, 42, 55

ReLU Rectified Linear Unit. 9, 10

RF Random Forest. xi, 3, 4, 9, 15, 16, 28, 33, 53

RNN Recurrent Neural Network. 4, 11

TSF Time Series Forecasting. 3, 4, 6, 15

List of Figures

5.1 High-level architecture of the code repository 18

6.1 Close price of AAPL 21
6.2 AAPL log-return analysis 22
6.3 AAPL log-volume analysis 22
6.4 AAPL daily volume 23
6.5 Data availability and split points of dataset 24
6.6 Boxplot of unscaled features 25
6.7 Boxplot of winsorized features 25
6.8 Boxplot of standardscaled and winsorized data 26
6.9 Boxplot of robustscaled and winsorized data 26
6.10 Learning curves 40
6.11 Comparison of predicted log-returns 41

List of Tables

3.1 Comparison of electricity demand and financial markets 7

6.1 Overview of feature complexity permutations and dimensions 28
6.2 Classification of selected models 30
6.3 Statistical results 35
6.4 Economic evaluation 36
6.5 Descriptive statistics of scaled features 39
6.6 Comparison of statistical properties: Ground truth vs. model predictions 41

1 Overview of all features 44
2 Hyperparameter tuning space and best parameters for Random Forest (RF) models across all horizons. 53
3 Hyperparameter tuning space and best parameters for Multi-Layer Perceptron (MLP) models across all horizons. 54
4 Hyperparameter tuning space and best parameters for Patch Time Series Transformer (PatchTST) models across all horizons. 55

1 Introduction

1.1 Motivation

Forecasting of financial markets has always been a very complex and nearly impossible task. In a time where algorithms determine most of the trading volume, the debate about the predictability of stock prices has taken on new dynamics. Driven by recent successes in artificial intelligence and large language models, this has also affected models in time series forecasting and introduced the use of increasingly complex forecasting models – from Long Short-Term Memory (LSTM) models to Transformer architectures [1] and modern state-space models like Mamba [2]. The literature around top-tier AI and ML conference papers has been increasing very fast in recent years [3, p. 4]. With newer, more complex models, there have always been new records in benchmark datasets to prove their superior state compared to previous models.

However, this focus may raise one question: Do these results justify the massive increase in computing power and the reduced interpretability of deep learning models? With the focus on new models, the importance of input data is maybe neglected. It is questionable whether a highly complex Transformer could achieve the same as a linear model with economically based feature engineering. Does the power of modern artificial intelligence models lie in actually understanding relationships in data better, or do they learn indicators that could already be obtained more efficiently through classical technical analysis? The answer to this question would also determine if domain knowledge is needed to train a well-performing model or if a profound knowledge of machine learning is required. There is already an ongoing discussion [4] as to whether simpler linear models can achieve the same as more complex models, at least in long-term time series forecasting in different domains.

1.2 Efficiency vs. Predictability

This conflict stands in direct contrast to established financial market theory. According to the EMH, all available information is already considered in current prices. This suggests that consistently outperforming the market is not possible [5]. This results in

the theory of a symmetric random walk, where price changes are random and therefore, in principle, unpredictable [6]. Nevertheless, empirical studies exist that show financial time series are not purely random signals. Technical analysis assumes that historical patterns will repeat and can be measured using different indicators [7].

1.3 Goal of the Thesis

The goal of this work is to quantitatively examine the relationship between model complexity and feature complexity. A quantitative analysis should provide answers to the following questions:

- Can complex models like a Transformer with minimal engineered input data achieve the same or better performance than simpler ones?
- Can feature engineering, with domain knowledge, resulting in complex technical indicators boost the performance of simpler models?

This should result in a decision on where more effort and computational resources should be invested, in feature engineering or in training a complex model.

1.4 Structure of the Thesis

The thesis is structured as follows: First, a review of the relevant literature is presented in Chapter 2. The underlying principles of financial market theory and the technical prerequisites for the utilized models and strategies are explained in Chapter 3. The methodology is described in Chapter 4. Chapter 5 provides an overview of the code repository and details, on how the pipeline was implemented in Python. A comprehensive data analysis and the experimental setup, including the results, are described in Chapter 6. The thesis concludes in Chapter 7 with a summary of the findings and an outlook on future research possibilities.

2 Literature Review

This chapter reviews the development of machine learning models and clarifies why skepticism towards more complex models is justified for time series forecasting. Furthermore, it also gives a short introduction to asset pricing in traditional finance literature.

Development of Machine Learning Models and Their Use for Time Series Forecasting (TSF): With the refinement of the MLP and the backpropagation algorithm [8], modern deep learning was initiated. They could model nonlinear patterns but lacked the ability to capture temporal dependencies in time series data.

Machine learning models like RF were published around the early 2000s [9]. They utilize an ensemble of trees which achieve significant improvements in predictions with the benefit of being robust with respect to noise. With the introduction of LSTMs, a novel approach was presented to preserve a cell state of previous inputs, allowing information to be maintained over longer periods [10].

The recent decade has been dominated by the rise of deep learning. Most notably the Transformer architecture, the model architecture behind Large Language Models (LLMs) like ChatGPT. Due to its success in capturing long-range dependencies, researchers have been applying the architecture to the domain of TSF.

During recent years, a massive increase in top-tier artificial intelligence and machine learning conference papers on TSF can be examined [3, p. 4]. The success of Transformer models for LLMs is undoubted, however, does this also count for the TSF domain? The increasing complexity and the need for more computational power with this architecture raise the question of whether these models are justified for that purpose. Zeng et al. [4] showed that a simple one-layer linear network (DLinear) outperforms or can achieve the same results as state-of-the-art Transformers for long-term TSF. This further underlines that simpler models are justified and should be reexamined.

With the introduction of PatchTST [11], this issue was addressed. It divides the input data into patches, which reduces the training time by a linear factor (in their experimentation setup by as much as 22 times on large datasets) while still outperforming pointwise Transformer baselines. Therefore, fundamental deep learning models are being looked at

again, like MLPs, Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Graph Neural Networks (GNNs), as well as Transformers overcoming their current limitations as in PatchTST.

New innovative models like Mamba try to reduce the quadratic time and space complexity of Transformers to linear scaling with context length [2] and new Diffusion models [12] are introduced. Although there is a trend towards simpler models, increasing popularity can be observed in foundation models [3, p. 82]. These models are already pre-trained on millions of data points and are, if at all, only fine-tuned on the specific dataset used. They already inherently learned structures from time series and have impressive zero-shot capabilities. This does help to remove task-specific training but still preserves a structurally complex model.

Use of Machine Learning in Financial TSF: For financial TSF, many papers exist on different financial instruments and forecasting strategies. Most publications exist on stock price forecasting and for a horizon of one day of daily returns [13, p. 18]. As Sezer et al. [13] stated, the implementation of deep learning prediction models led to many new publications in the financial community.

Random Forests and deep neural networks were already applied to forecasting daily returns of the S&P 500, which resulted in Random Forest as the model of choice [14].

Kraus et al. [15] later proved in their work the advancements of the LSTM model beyond other machine learning models such as RF and AutoRegressive Integrated Moving Average (ARIMA) models. It resulted in profitable prediction strategies prior to transaction costs for LSTM models.

As Sezer et al. [13] concluded in their literature review, in most of the studies, deep learning outperformed traditional machine learning methods, but still in many cases their performance was comparable [13, p. 25].

Financial Literature: The development of asset pricing models represents the increasing recognition of market factors. Starting with the Capital Asset Pricing Model (CAPM) [16], a single-factor model assuming that the expected return of the asset is only determined by β, which measures the relationship between the asset and the overall market.

The Fama-French Three-Factor [17] model extends the CAPM by two additional factors: the size factor and the value factor. The size factor is based on the observation that small-cap stocks tend to exhibit higher returns than large-cap stocks, while the value factor assumes that assets with a high book-to-market ratio outperform ones with a lower ratio.

With the introduction of the Carhart Four-Factor Model [18], the model was further extended by a momentum factor. This factor also incorporates behavioral finance theories, such as the tendency of investors to buy well-performing assets and sell poorly performing assets [19].

3 Technical Prerequisites

3.1 Definitions and Forecasting Strategies

Time series analysis involves the study of data points indexed in temporal order. In the context of financial markets, the objective is to utilize historical price data to predict future values y_{t+h}, where h represents the predicted horizon and t is the last timestamp of observed data.

Univariate and Multivariate TSF: Depending on the input dimensionality, it is distinguished between univariate TSF, which uses only the target variable's history and multivariate TSF, which incorporates exogenous variables (features).

Short-Term TSF and Long-Term TSF: Short-Term TSF: Typically focuses on the next immediate steps ($h \in \{1, \ldots, 10\}$) whereas Long-Term TSF focuses on modeling underlying trends over extended horizons. In recent state-of-the-art literature and their utilized benchmarks [11], Long-Term TSF is often defined by horizons of $h \in \{96, 192, 336, 720\}$.

Direct and Recursive Forecasting: An important consideration is, how a model handles multi-step predictions ($h > 1$).

- Recursive (Iterative) Forecasting: The model only predicts the next timestamp $t + 1$ and uses this prediction as the next input to predict $t + 2$. This technique is less robust for large horizons, since inaccuracies in the first predictions accumulate through the entire horizon window.

- Direct Forecasting: The model is trained to predict the specific horizon $t + h$ directly from t. This is more robust for longer horizons, as it avoids the error propagation of recursive models.

3.2 Forecasting in the Financial Domain

The predictability of a specific time-series domain is dependent upon several structural factors. According to Hyndman and Athanasopoulos [20], the possibility of forecasting depends on how well the contributing factors can be understood, the amount of data available, and whether the forecast itself affects the system.

Financial markets, particularly stock prices, present a unique challenge when compared to more "stable" domains like electricity demand. Table 3.1 illustrates these fundamental differences. As demonstrated in the comparison, the financial domain is characterized

Factor	**Electricity Demand**	**Financial Markets (Stocks)**
Understanding factors	High (Weather, time of day)	Low (Market sentiment, news, economic factors)
Data availability	High (smart meters)	High (ticks), but high noise-to-signal ratio
Future vs. Past	Very similar (seasonal)	Often dissimilar (structural/ regime shifts)
Reflexivity	Low (demand is passive)	High (self-fulfilling prophecies)

Table 3.1: Comparison of electricity demand and financial markets

by a high degree of reflexivity. This suggests that forecasting can alter the variable being measured. If a widely used model predicts an upward trend in a stock's price, the resulting buying pressure from market participants may cause the price to rise, thereby validating the model through a self-fulfilling prophecy, rather than by accurately capturing the underlying fundamentals.

Because raw stock prices are highly non-stationary, literature suggests using returns. Specifically, logarithmic returns come with two primary advantages. First, log-returns result in time-additivity, meaning that multi-period return can be expressed as the sum of the sub-period returns. For an asset, the log-return over a horizon h can be written as:

$$y_{t,h} = \sum_{i=1}^{h} r_{t+i} = \ln(P_{t+h}) - \ln(P_t) \tag{3.1}$$

where $y_{t,h}$ represents the cumulative log-return from time t to $t+h$, r_{t+i} represents the log-return at each step. P_t the asses price at time t and P_{t+h} the asset price at time $t+h$.

Second, log-returns tend to exhibit statistical properties that are closer to a normal distribution. Unlike simple returns, which are bounded from below by -100%, the theoretical range of log-returns $y_{t,1}$ is theoretically unbounded:

$$y_{t,1} \in (-\infty, +\infty) \tag{3.2}$$

3.3 Prediction Models

The following model architectures are used for predictions and are further explained.

3.3.1 Linear Regression

A linear regression model can model linear dependencies between input variables. Ordinary Least Squares (OLS) aims to predict a dependent variable y through a linear combinations of k independent input variables $\mathbf{x} = [x_1, x_2...x_k]$ at each timestamp t. This is represented in the following equation:

$$y_t = \beta_0 + \beta_1 x_{t1} + \beta_2 x_{t2} + \cdots + \beta_k x_{tk} + \epsilon_t \tag{3.3}$$

where β_0 represents the intercept, β_j are the coefficients (weights) for each feature and ϵ_t denotes the error term representing variance that cannot be explained by the independent variables.

The learning process works with minimizing the Residual Sum of Squares (RSS) over the entire training set. This leads to the following loss function.

$$RSS(\beta) = \sum_{t=1}^{n} (y_t - \hat{y}t)^2 \tag{3.4}$$

The objective is to determine β such that the distance between ground truth y and the predicted value $\hat{y}$ is minimized across the whole training set.

3.3.2 Random Forest

A RF is an ensemble learning method that creates multiple estimators (decision trees) during training. It uses bagging (Bootstrap Aggregation) in combination with random feature selection. It divides the feature space into smaller chunks that are sampled randomly. A decision tree splits data recursively. At each node the algorithm splits data based on a feature j and a split-point s that minimizes the impurity. The impurity is typically measured as the Mean Squared Error (MSE) - at least for regression trees. In this context the MSE is the equivalent to minimizing the within-node variance. Therefore the main objective for a split-node is to minimize the variance of the child nodes.

$$L(j, s) = \frac{N_L}{N_t} MSE_L + \frac{N_R}{N_t} MSE_R \tag{3.5}$$

where N_t is the number of samples at the current node, and N_L, N_R are the number of samples in the left and right child nodes respectively. MSE_L and MES_R represent the errors of the left and right branch after the split respectively. A single decision tree is prone to overfitting (high variance). A RF mitigates this problem with two mechanisms.

- Bootstrapping: A random subsample of the dataset is drawn with replacement. That means observations can occur in a single tree multiple times and some can be left out

- Feature Subsampling: When spliting on a criterion this is also chosen from a randomly subsampled set of all features.

The final prediction $\hat{y}$ for an input x is obtained by averaging the predictions of all M individual trees in the forest:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^{M} h_m(x) \tag{3.6}$$

where $h_m(x)$ represents the prediction of the m-th individual tree. The ensemble method makes this estimator more robust to outliers and noise, which are especially present in financial time series data. Since predictions are based on the learned splits from the training sets, a RF can only interpolate from previously seen values.

3.3.3 Multilayer Perceptron

The MLP is considered a deep neural network, that consists of one input layer, one or multiple hidden layers and an output layer. Because of non-linear activation functions like Rectified Linear Unit (ReLU) within the hidden layers, it is able to model complex

non-linear relationships in the input data.

- **Input Layer**: Receives the preprocessed features $\mathbf{x}$.
- **Hidden Layers**: Fully connected layers that transform data into higher-level representations using non-linearities.
- **Output Layer**: Produces the final prediction $\hat{y}$. For a regression task, the number of neurons in the output layer corresponds to the number of predicted horizons.

The fundamental building block of an MLP is the perceptron. A single perceptron represents a function of different input parameters, weights and a bias. This can be represented the following:

$$z = \mathbf{w}^T \mathbf{x} + b \tag{3.7}$$

where z is a single value, the output of the perceptron, $\mathbf{x}$ the input vector, $\mathbf{w}$ the weight vector for each input value respectively and b a bias term.

The input layer represents the input features, a hidden layer consists of multiple perceptrons (units). Each unit in that layer calculates a weighted sum of the inputs and applies a nonlinear activation function to it. The activation of a hidden unit z_h can be defined as following:

$$z_h = \sigma \left(\sum_{j=1}^{d} w_{hj} x_j + w_{h0} \right), \quad h = 1, \ldots, H \tag{3.8}$$

x_j describes the input features ($j = 1, ..., d$). w_{hj} the weights of the connection from input j to hidden unit h. w_{h0} the bias weight of the hidden unit. σ the non-linear function like a sigmoid function or ReLU. The output layer takes those activations z_h of the last hidden layer as input. For a regression problem, the output y_i is typically a linear combination of the hidden units.

$$y_i = \sum_{h=1}^{H} v_{ih} z_h + v_{i0}. \tag{3.9}$$

v_{ih} represents the weights of the last hidden layer (from hidden to output) and v_{i0} the bias of the output layer.[21, pp. 298-299] This allows the MLP to predict continuous values that are not limited by any activation function.

3.3.4 Transformer Architecture

Instead of RNNs, which process data sequentially, the attention mechanism in Transformers allows the model to look at all data points in a sequence simultaneously. This lets it capture relationships between any two positions in the input sequence.

The core of the Transformer is the **Scaled Dot-Product Attention**. This mechanism consists of three learnable weight matrices:

- **Query (Q):** Represents the current item for which the model is seeking information.
- **Key (K):** Acts as a reference, to help the model calculate similarity scores.
- **Value (V):** Contains the actual information for each item in the sequence.

Mathematically, the attention score calculation was proposed by Vaswani et al. [1] as following:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{3.10}$$

- QK^T calculates the dot-product (similarity) between the query and all keys
- $\sqrt{d_k}$ is a scaling factor preventing extreme values from occuring with large dimensions d_k
- *softmax* function normalizes these scores into probabilities that sum up to 1, representing the attention weights
- Multiplying by V results in a weighted sum of the information

Multi-Head Attention (MHA) repeats the attention mechanism multiple times, allowing each head to learn individual representations from the data. The output of each head is concatenated and multiplied by a learnable weight matrix W^O and the number of attention heads h:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{3.11}$$

Since the attention mechansim does not keep the temporal order, a positional encoding

vector is added to the input embeddings. For Transformers utilized as LLMs they typically follow an encoder-decoder structure, whereas in regression analysis an encoder-only architecture is sufficient to extract predictive features from time series data.

4 Methodology

The objective of this study is to compare the performance between different model and feature complexity levels C_k. The target for each model is to forecast the logarithmic return for future time horizons. Each model will then be evaluated based on machine learning metrics and for economic evaluation.

4.1 Forecasting Strategy

For each timestamp t, the models are designed to calculate the log-return for a set horizons $H \in \{h_1, h_2, \ldots, h_i\}$.

4.2 Global Model Approach

The models are implemented as global models, i.e. each model is trained on on the pooled data of all N tickers, where $n \in \{1, \ldots, N\}$ represents the individual asset. This allows for a much larger sample size for the training step. This approach also enables the individual models to learn cross-sectional patterns that can be generalized across different assets.

4.3 Target Variable Calculation

The target variable is the multi-period log-return. For each ticker n, the log-return $y_{t,n,h}$ for a horizon $h \in H$ is defined as the difference between the logarithm of the future close price $P_{t+h,n}$ and the close price at the current time $P_{t,n}$:

$$y_{t,n,h} = \ln(P_{t+h,n}) - \ln(P_{t,n}) \tag{4.1}$$

4.4 Feature-Complexity Concept

Feature complexity plays a major role in this thesis. The features are divided into three distinct complexity levels:

- **Level 10 – Primitive Features:** These represent first-order transformations of raw price and volume data. They provide a direct mapping of the market Open, High, Low, Close, Volume (OHLCV) data and serve as the raw baseline.
- **Level 20 – Intermediate Features:** These comprise properties of the volume or price distribution or incorporate temporal patterns. They consist of second-order statistics or time-domain encoding.
- **Level 30 – Advanced Features:** These are "features of features" or complex oscillators. They calculate relationships between different averages or normalize price movements within a range. They represent compact information to express momentum, trend strength, or mean-reversion signals.

A higher complexity level signifies that the feature involves more abstract mathematical transformations, which theoretically provides the model with information-dense features, whereas Level 10 represents raw or near-raw baseline data.

With this division into three different levels, resulting in seven unique complexity combinations C_k, it can be shown if more complex features result in better model performance and if a simple linear model can already perform well on highly complex features. Whereas complex models might have the ability to create complex features themselves.

Description of Feature Categories: In the context of machine learning, features serve as the foundation of a models predictive power. In the financial domain, these features are primarily referred to as technical indicators. Using these indicators for forecasting price movements is known as technical analysis. Technical indicators can be categorized based on the information they represent [7]. An important aspect of these indicators is their temporal information. Unlike static data, financial indicators are calculated over different time horizons and therefore represent the time history into the current input vector. This allows the model to capture different dynamics in the sequence.

Trend and Directional Movement Indicators: These indicators are designed to identify the direction of the market by smoothing out noise from short-term price movements. They rely on the assumption that a trend is more likely to continue than to reverse [22] A typical trend indicator is the moving average.

Oscillators and Momentum Indicators: Momentum measures the velocity of price changes. Osciallators are a sub-group of momentum indicators and are typically set to a fixed range (e. g. 0-100). They are utilized to signal a overbought or oversold condition [23]. A prominent momentum indicator is the Relative Strength Index (RSI).

Volatility-Based Features: Volatility features measure the magnitude of price movements. They indicate how much the price has moved up or down. They measure the volatility and therefore the risk of the underlying asset.

Volume and Liquidity Features: Volume represents the amount of traded shares over a time horizon. It indicates whether a price movements is strong and if it is supported by market activity. It shows the market liquidity and can validate price breakouts.

Statistical and Temporal Features: These features describe mathematical properties of the return. They are used to represent the Kurtosis of the distribution of returns over a past horizon. Additional temporal features are used to learn potential seasonal patterns.

4.5 Model-Complexity Concept

In order to evaluate the relationship between feature complexity and model performance it is necessary to form a taxonomy of the models used in this study. As noted by Hu et al. [24, p.3] model complexity cannot always be directly compared across different model frameworks. However the properties of individual architectures can be used to classify each estimator into a hierarchical structure:

- **Functional Linearity:** A linear model assumes that the relationship between features $\boldsymbol{x}$ and the target y can be expressed as a weighted sum. Non-linear models utilize activation functions or recursive splitting to approximate non-linear mappings.

- **Architectural Parametrization:** This differentiates between parametric models which have a fixed structure of learnable weights regardless of the training set size and non-parametric models. In a RF the complexity of the individual trees and splits is dependant on the available data, which makes it non-parametric.

- **Temporal Attention** This distinction is crucial in TSF. Pointwise (time-agnostic) models treat each input vector as an independant data point. Structural (context-

aware) models, can process multiple data points at once and therefore examine temporal dependencies across the sequence.

4.6 Overview

To provide a better understanding, the process of the methodology is explained using the defined notation. For each ticker n, its input data at time step t is represented by a feature vector $\mathbf{x}_{t,n} \in \mathbb{R}^D$, where D represents the number of dimensions corresponding to the chosen complexity level C_k.

Since each estimator is trained on all ticker data, the individual feature vectors are concatenated into a global feature matrix $\mathbf{X}_t = [\mathbf{x}_{t,1}, \mathbf{x}_{t,2}, \ldots, \mathbf{x}_{t,N}]^T \in \mathbb{R}^{N \times D}$. A specific estimator f is trained to predict the vector of log-returns for the predefined horizons H. Although the feature matrix has the format $\mathbf{X}_t$, the estimator only processes the data of one ticker n at a time. The mapping can be written as:

$$\hat{\mathbf{y}}_{t,n} = f(\mathbf{x}_{t,n}) \tag{4.2}$$

where $\mathbf{x}_{t,n}$ is the feature vector of dimension D. The output $\hat{\mathbf{y}}_{t,n} \in \mathbb{R}^i$ is a vector representing the predicted log-returns for i predicted horizons respectively:

$$\hat{\mathbf{y}}_{t,n} = \begin{bmatrix} \hat{y}_{t+h_1,n} \\ \hat{y}_{t+h_2,n} \\ \cdots \\ \hat{y}_{t+h_i,n} \end{bmatrix} \tag{4.3}$$

For the baseline models, a multi-output wrapper is utilized, allowing the linear regression and RF architectures to map the input features to the vector $\hat{\mathbf{y}}_{t,n}$ by optimizing for each horizon simultaneously.

5 Implementation

5.1 Pipeline Definition

The experimental setup is implemented in Python, using a modular pipeline architecture designed for customizable operations. This chapter details the technical implementation to provide a comprehensive understanding of the workflow of experimental framework and ensure the reproducibility of the results. The Code repository is available at: `https://git.fiw.fhws.de/k62214/financial-time-series-forecasting`. The pipeline is built by the principles of separation of concerns, ensuring that each pipeline step remains isolated and modular. The pipeline class (see Appendix 2) acts as a scaffold for the setup, upon instantiation, the pipeline object initializes the required steps and the corresponding model class from the configuration (see Appendix 3). Communication between steps is handled via a standardized `StepResult` object, which carries data and metadata through the sequence. Each individual step implements a unified `PipelineStep` interface (see Appendix 4), enforcing a consistent structure throughout the execution process.

5.2 Configuration-Driven Execution

For a consistent experimental setup, the entire workflow is managed by a central configuration file, `config.json` (see Appendix 1). This file defines the parameters and configurations for the execution environment, including:

- **Data Configuration**: Defines the relative paths for OHLCV data, the specific date range, and the number of tickers to be included.
- **Model Configuration**: Specifies the machine learning architecture to be initialized for the current setup.
- **Feature Configuration:** Points to a `feature_config.json` and defines which complexity levels are active.

- **Pipeline Configuration**: Lists the sequential order of the pipeline steps to be executed.
- **Results Configuration**: Sets the relative path for result logging and saving the fitted model.
- **Horizon Settings**: Defines a configurable list of target horizons H for the multi-step prediction task.

An overview of the code structure can be seen in Figure 5.1.

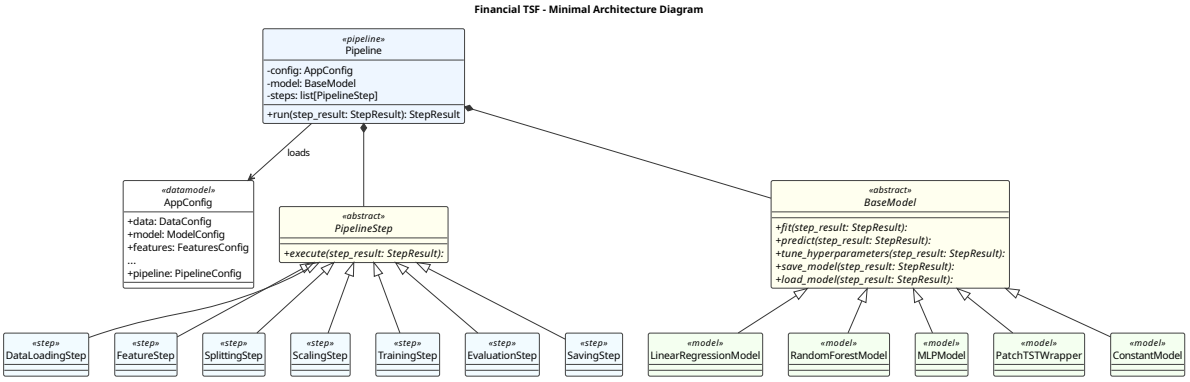


Figure 5.1: High-level architecture of the code repository, highlighting the abstract base classes and their concrete implementations.

5.3 Pipeline Execution Workflow

The pipeline iterates through the defined steps sequentially. Each step performs a specific task, storing the results in the `StepResult` object for the next step:

`DataLoadingStep`: This step loads all CSV files containing the OHLCV data based on the configuration. Data validation is performed on each ticker individually to ensure consistency across the selected date range.

`FeatureStep`: This step computes the technical indicators defined in the `featureconfig.json` (Appendix 5). This configuration also acts as the mapping of features to their respective complexity levels. Only features corresponding to the active complexity levels from the file are calculated.

`SplittingStep`: This step determines the global splitting timestamps based on the ticker with the largest available date range (for training, validation and testing datasets).

`NormalizingStep`: This step implements a two-stage preprocessing setup. First **Winsorization** (clipping) is applied to the features based on the training set distribution. Second, a `RobustScaler` is fitted on the training data exclusively and applied to the validation and test sets to prevent data leakage.

`TrainingStep`: This step manages the models training and fitting process. If enabled, hyperparameter optimization is performed to identify the best configuration using the validation set. Next, the model is fitted on the training data either with a default set of hyperparameters or using the best-performing set from the previous optimization run. Using the fitted model, all specified horizons are predicted on the test set. Since all models inherit from a common base interface, the execution logic remains uniform across all architectures (see Appendix 3).

`EvaluationStep`: This step reverses the scaling of respective data using per-ticker metadata and maps the global prediction dataset back to specific tickers. It computes multi-step performance metrics for both statistical evaluation and economic analysis.

`SavingStep`: This step saves all experimental components including configurations, trained models, metrics and raw predictions into a folder created specifically for the execution. A summary text file concludes the most important aspects of the run stating the corresponding feature list, sample counts, splitting points, training durations and the individual metrics.

6 Experiments

6.1 Data Analysis

6.1.1 Data Selection

The Nasdaq-100 Index was chosen as the data source. The composition of the index was retrieved from the Nasdaq website (`https://www.nasdaq.com/market-activity/quotes/nasdaq-ndx-index`, accessed on January 2, 2026). The observation period ranges from 2015-01-01 to 2023-12-31 with a 5-minute trading interval during regular trading hours (9:30 a.m. to 4:00 p.m.), Monday through Friday. Historical data was fetched from `AlphaVantage` for all 101 Tickers. The historical data is only very limited available with a free membership. For reproducibility an own API key is required. The available data for each ticker is OHLCV data, consisting of Open, High, Low and Close prices as well as volume (number of traded shares) for each 5-minute interval. Tickers with limited data availability during the specified time interval, due to different Initial Public Offerings (IPOs), are used starting from their first available trading date.

6.1.2 Statistical Analysis

In financial data, certain domain-specific statistical properties must be adressed before approaching further analysis. A primary issue with raw ticker prices is that they are non-stationary, meaning statistical properties like the mean and variance change over time. This is illustrated in Figure 6.1. Specifically neural networks which were trained on non stationary data will not learn generalizable patterns but specific trends of the training period.

The log-returns are calculated relative to the previous close price. After applying this transformation, the following plot can be examined (Figure 6.2a). The log-returns follow a relative stationary nature.

Another property of financial returns is that their distribution typically follow a distribution with fat tails and high kurtosis compared to a normal distribution (Figure

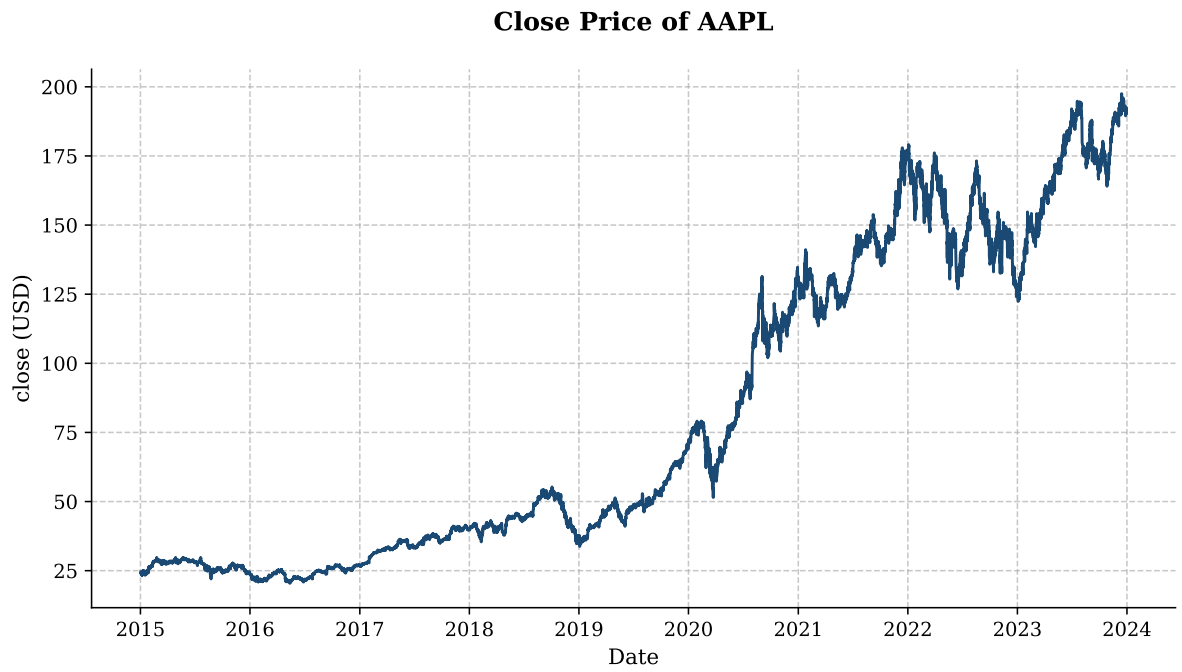
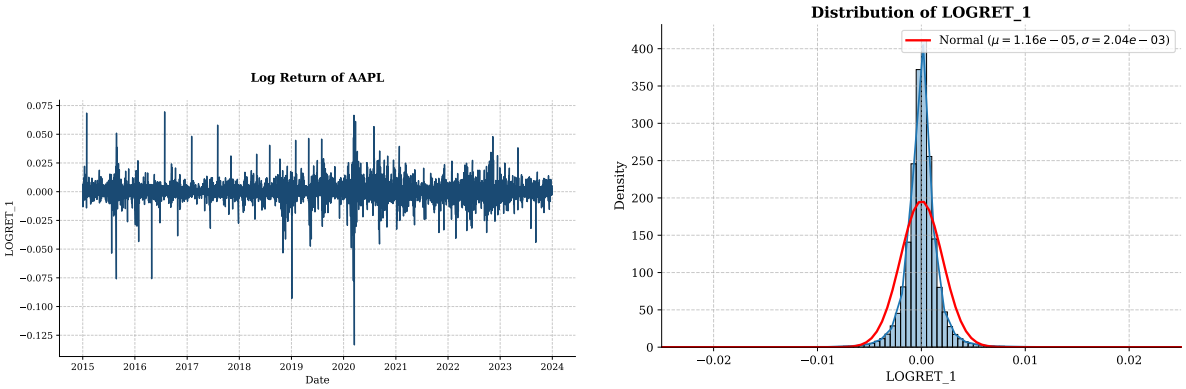


Figure 6.1: The Close price in USD of AAPL over 5-minute intervals during the selected observation period

6.2b).

Analyzing the volume reveals significant deviations from a normal distribution. As illustrated in the raw volume distribution (see Figure 6.3a), the data reveals a strong positive skewness. The high variance relative to the mean highlights the presence of extreme outliers. These outliers are primarily caused by the intraday dynamics of the stock market. As shown in the intraday volume graph (Figure 6.4), the volume represents a U-shaped pattern. Significant spikes during the market opening at 09:30 a.m. and the market close at 4:00 p.m. can be observed. To reduce the impact of outliers, a log-transformation was applied. This transforms the distribution into a more bell-shaped curve (see Figure 6.3b)

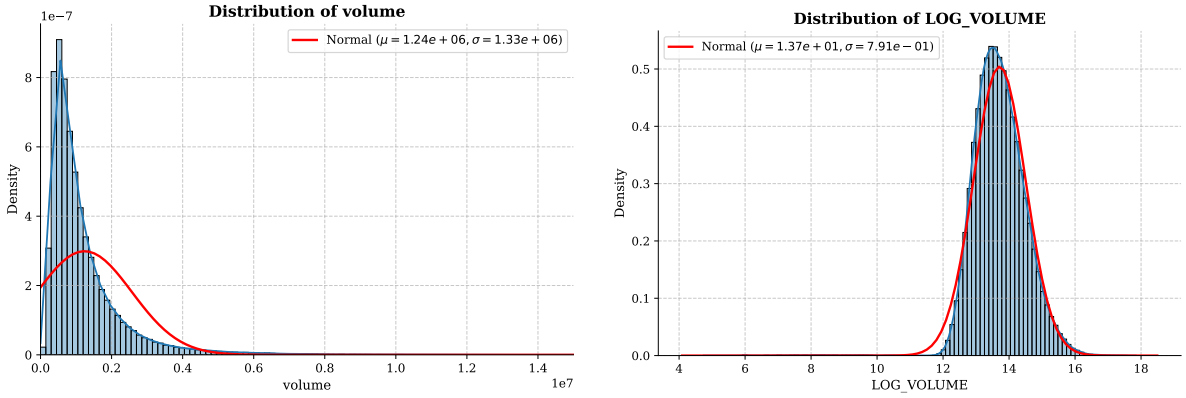
The analysis for returns was performed only on the close price of the 5-minute interval. The open, high, low, and close prices are highly correlated, especially in short intervals like 5-minutes and would result in nearly the exact same statistical properties and distributions.



(a) Time series of returns.

(b) Distribution analysis.

Figure 6.2: Analysis of 5-minute log-returns for AAPL. **Left:** The plot illustrates the stationarity of the returns and the presence of volatility clustering throughout the dataset period. The log-return fluctuates around 0, although some extreme outliers can be observed. **Right:** Distribution of 5-minute log-returns for LOGRET_1 (The log-return of previous 5-min interval). The plot compares the log-return distribution against a theoretical normal distribution with corresponding μ and σ of the distribution.



(a) Distribution of volume.

(b) Distribution of log-volume.

Figure 6.3: Analysis of 5-minute log-volume for AAPL. **Left:** The time series shows log-volume where slight distribution shifts can be observed. **Right:** The distribution of log-transformed volume compared to a normal distribution with the properties of the underlying dataset (μ, σ).

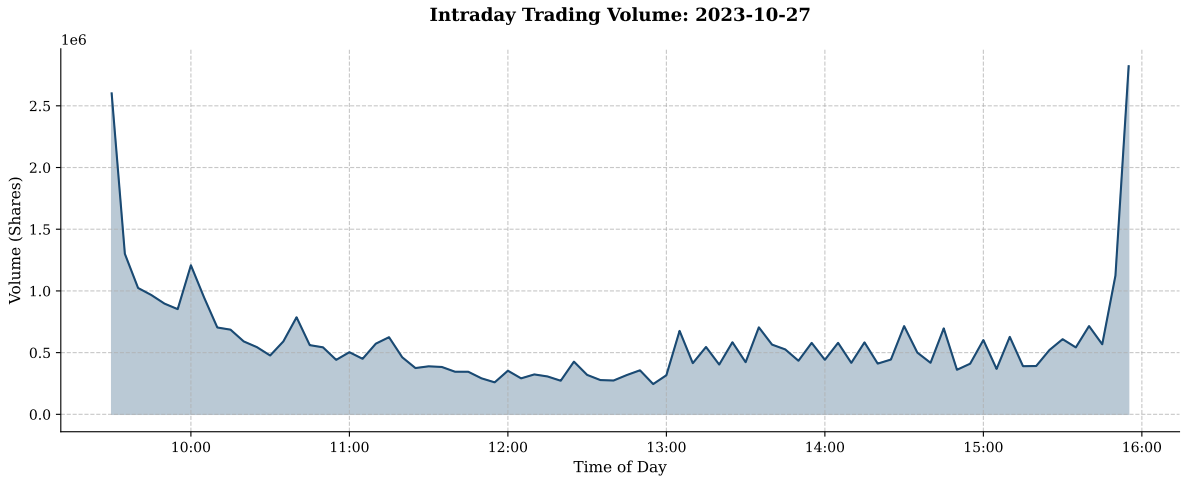


Figure 6.4: AAPL volume for a randomly selected day (2023-10-27). Volume spikes at the opening and closing times of the stock exchange can be observed.

6.1.3 Data Partitioning

Following the feature engineering and target generation process, the dataset consists of complete feature-target pairs for each timestamp across all tickers. To evaluate the model performance and generalization capability, the data is partitioned into three distinct subsets: training, validation, and testing. This step is done chronologically to respect the temporal order of financial data, avoiding any look-ahead bias. The partition ratios are defined as follows:

- **Training Set (75%)**: Selected to optimize the model parameters (weights) during learning process.
- **Validation Set (15%)**: Used for hyperparameter tuning and to prevent overfitting with early stopping during the training of neural networks.
- **Testing Set (10%)** Reserved for the final out-of-sample evaluation. This set is used to calculate model performance metrics and economic evaluation.

Since the dataset comprises multiple tickers with different listing dates, a per-ticker split would result in inconsistent time splits (e.g. the training period for Ticker 1 might overlap with the testing period of Ticker 2). To prevent this, a global time-split strategy was implemented across all tickers. The ticker with the longest historical data serves as the reference to determine the absolute calendar timestamps for the split points. All tickers are then split based on these global split points. This ensures that the model is never trained on future market regimes of other assets. With this strategy applied, it is possible for individual tickers to have missing data because they were not listed at that

time. Tickers that did not comprise at least two years of available data in the training set were excluded from the study. This exclusion is justified because the model cannot learn the specific patterns of the ticker during training and would therefore generalize poorly in testing. Applying this strategy reduced the number of assets from 101 to 92

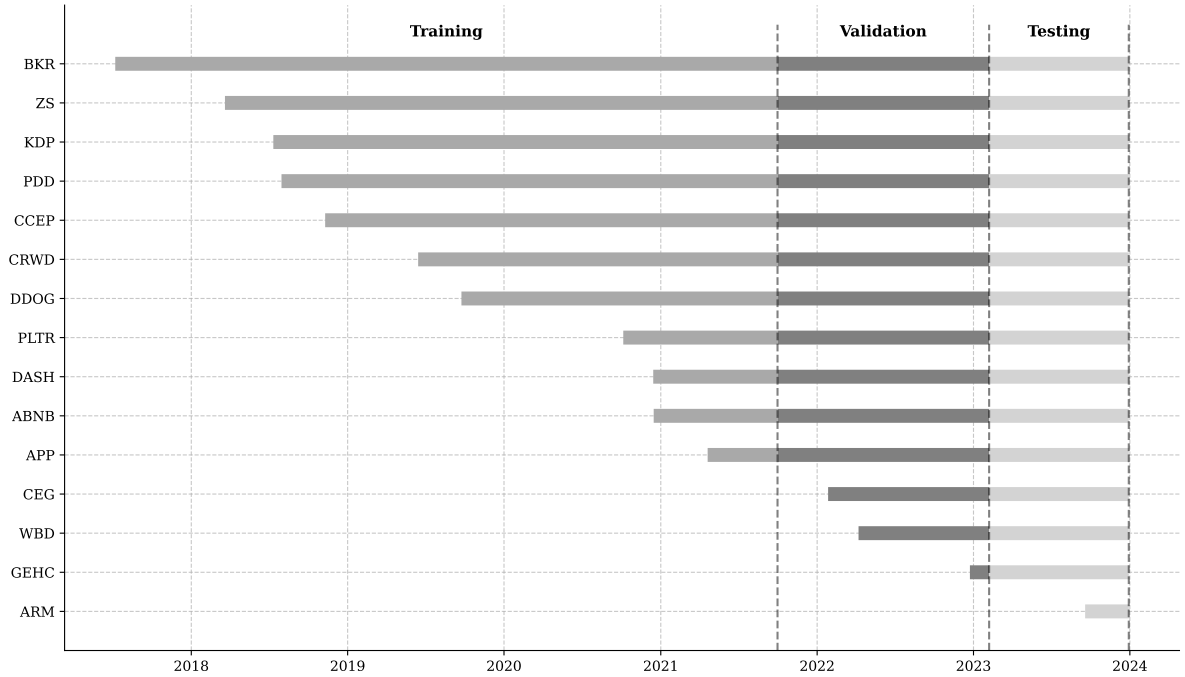


Figure 6.5: Data availability of the top 15 tickers sorted by their first available timestamp in descending order. Start point for the validation set: 2021-09-30. Start point for the testing set: 2023-02-06.

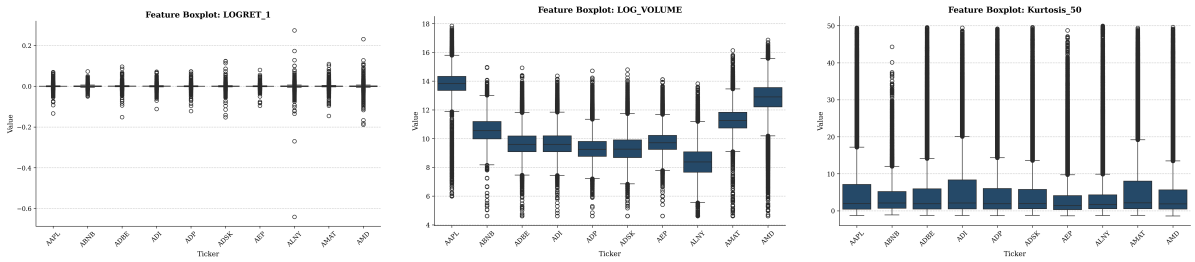
tickers. Figure 6.5 illustrates the data availability and the global split lines. In this case, out of 101 tickers, 9 were removed after applying this strategy.

6.1.4 Preprocessing

Handling Outliers

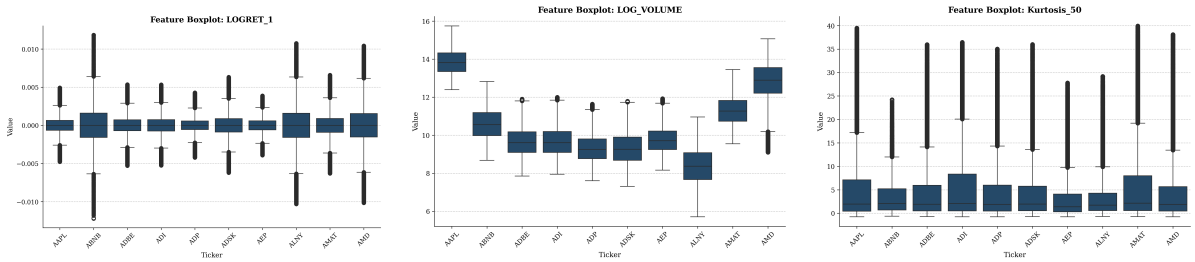
Since log-returns in financial time series follow leptokurtic distributions as previously discussed, a multi-stage preprocessing pipeline is applied. To prevent extreme values from distorting the parameters of the subsequent scaling process, the input features are first winsorized. For each ticker n, all feature values below the 1st percentile and above the 99th percentile are capped at these thresholds. This approach aims to reduce the impact of extreme outliers in the dataset while retaining the underlying information. The distribution of three example features is first visualized without any preprocessing

(see Figure 6.6) and then after winsorization has been applied (see Figure 6.7). This comparison shows the impact of outliers on the data distribution.



(a) Boxplot for log-return. (b) Boxplot for log-volume. (c) Boxplot for kurtosis.

Figure 6.6: Boxplot of unscaled features across 10 selected tickers. It can be observed that the range of the features varies significantly and extreme outliers are present.



(a) Boxplot for log-return. (b) Boxplot for log-volume. (c) Boxplot for kurtosis.

Figure 6.7: Boxplot of winsorized features using the 1st and 99th percentiles as thresholds across 10 selected tickers. In 6.7a, it can be observed that the range of log-returns is reduced drastically compared to the unscaled version in Figure 6.6a. These extreme outliers have been capped at the threshold values.

Scaling

The next step is to center and scale the data of each ticker n respectively. A common approach in machine learning is standardization (Z-score normalization), defined as:

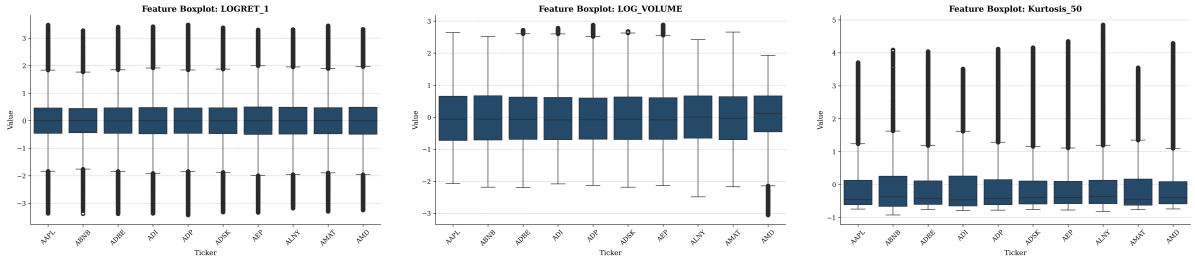
$$x_{\text{standard scaled}} = \frac{x - \mu}{\sigma} \tag{6.1}$$

where μ represents the mean and σ the standard deviation. However, as some outliers are still present even after winsorization, the mean is affected by them. It is pulled too strongly towards the extreme values. To avoid this problem, robust scaling is used. This method utilizes the median and the Interquartile Range (IQR):

$$x_{\text{robust scaled}} = \frac{x - \text{median}}{Q_3 - Q_1} \tag{6.2}$$

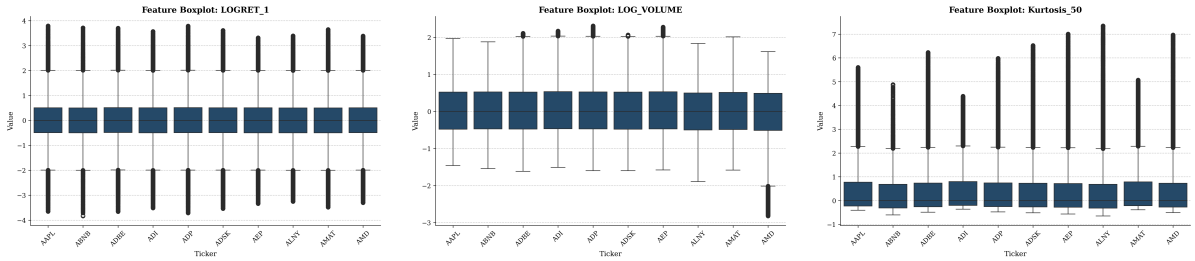
Since the IQR (the range between the 25th(Q_1) and 75th(Q_3) percentiles) only considers the central distribution measure, the scaling factor is significantly more stable across different market phases and volatility jumps.

It can be observed that the median for the feature kurtosis with the standard scaling method (Figure 6.8c) is distorted more into the negative direction compared to the robust scaling method (Figure 6.9c). This is the result of the still present positive outliers.



(a) Boxplot for log-return. (b) Boxplot for log-volume (c) Boxplot for kurtosis

Figure 6.8: Boxplot of data that was first winsorized and then scaled with the standard scaling method on selected tickers.



(a) Boxplot for log-return. (b) Boxplot for log-volume (c) Boxplot for kurtosis

Figure 6.9: Boxplot of the data that was first winsorized and then scaled with the robust scaling method on selected tickers.

Range Clipping

Despite robust scaling, individual features can still take on values outside the expected range (see Figure 6.9c) due to extreme volatility. Therefore a final "hard clipping" is performed. This limits all features of each ticker n to the interval $[-5, 5]$. The choice of this range is based on testing where the different models were most stable during training.

Data Leakage

To avoid data leakage, all parameters, the percentiles for winsorization, the median and IQR for robust scaling were calculated exclusively based on the training set and then applied to the validation and testing sets. This ensures that no future information is leaked into the scaling logic of the training phase.

Handling Target Variables

The target variables, the log-returns for the selected horizons, were only scaled, but not winsorized nor clipped. This approach aims to preserve actually price jumps or crashes, which the model should be able to predict.

6.2 Baselines

6.2.1 Overview of All features

All the engineered features are categorized into three hierarchical complexity levels. A distinction must be made between a **base indicator** and the resulting **input dimensions**, since many indicators depend on variable lookback horizons, a single base indicator can yield multiple input features. For instance, a Simple Moving Average (SMA) calculated using lookback periods of 20, 50, and 200, results in three distinct feature columns (input dimensions).

To evaluate the predictive power of the complexity levels each combination is tested, resulting in seven different combinations overall. Table 6.1 summarizes the distribution of base indicators and their corresponding total feature dimensions across the seven defined complexity permutations. A comprehensive list detailing each feature and its explanatory value can found in Appendix 1.

This approach allows for a detailed analysis of how different types of information contribute to the models forecasting accuracy. It also shows potential redundancies and synergies within the feature sets.

Table 6.1: Overview of feature complexity permutations and dimensions

Complexity Def.	Complexity Levels	Base Indicators	Total Dimensions
C_{L10}	10	3	8
C_{L20}	20	12	28
C_{L30}	30	13	14
$C_{L10+L20}$	10 + 20	15	36
$C_{L10+L30}$	10 + 30	16	22
$C_{L20+L30}$	20 + 30	25	42
$C_{L10+L20+L30}$	10 + 20 + 30	28	50

6.2.2 Models

The following models were selected for this evaluation and were classified according to the proposed models properties. They cover a range from traditional statistical estimators to state-of-the-art deep learning architectures.

Linear Regression

As the simplest model for this evaluation, a linear regression model is chosen to serve as a predictive baseline estimator. More specifically an OLS regression is implemented. It can model linear relationships between the input features and the target returns.

The primary advantage of this estimator is its computational efficiency and high interpretability. Its limitations are the lack of ability to capture non-linear relationships or temporal dependencies between the variables.

Random Forest

The RF recursivly splits the feature space, which allows it to model non-linear dependencies. The RF is a non-parametric model, meaning its complexity is dependent on the training data distribution instead of a fixed set of weights.

Although the RF can capture complex relationships between features at a specific timestamp, it treats each observation as an independent instance. This results in the model ignoring the sequential order or temportal context in the time series.

MLP

The MLP represents the first model in this analysis incorporating deep learning. It can model non-linear dependencies with its activation functions, allowing it to build complex relationships within the input data. The structure of the network is set before the learning process and during it, only the weights are adjusted accordingly. Like the previous estimators, the model is also context unaware. A flattened vector at time t is used for prediction also ignoring the temporal context in the sequence.

PatchTST

Representing the current state-of-the-art in long-term time series forecasting, the Patch Time Series Transformer (PatchTST) was chosen [11].

It utilizes two unique concepts compared to traditional Transformers, dividing the input data into patches and handling multivariate time series independently.

- **Patching:** Instead of embedding each individual data point, patches are used. A patch consists of a sequence of a set length, capturing multiple data points. This allows the model to capture local semantic information within each patch. Patchwise embeddings instead of pointwise embeddings help to reduce the computational complexity from $O(L^2)$ to $O((L/S)^2)$, where L is the sequence length and S is the stride, the non-overlapping part of the input patch.

- **Channel-Independence:** The model handles each feature channel as an univariate time series but they share the same weights in the encoder. This allows the model to learn temporal pattern across channels without beeing corrupted by the high dimensionality of the original multivariate signal.

PatchTST utilizes only the encoder part of the traditional Transformers, in which each embedding is transformed into an abstract representation. These representations are then used in a regression head, which in this case is responsible for forecasting the different time horizons.

The self-attention mechanism allows the estimator to learn dependencies across different timesteps within the lookback window.

6.2.3 Model Categorization Matrix

The Table 6.2 displays the classification of each model.

Table 6.2: Classification of selected models

Model	Linearity	Parametric	Temporal Awareness
Linear Regression	Linear	Yes	No (Pointwise)
Random Forest	Non-Linear	No	No (Pointwise)
MLP	Non-Linear	Yes	No (Pointwise)
PatchTST	Non-Linear	Yes	Yes (Context-aware)

6.2.4 Complexity Hierarchy Justification

1. **Level 1 - Linear Regression:** Assumes global linearity and serves as the baseline.

2. **Level 2 - Random Forest:** Introduces non-linear decision boundaries through recursive data splits.

3. **Level 3 - MLP:** Utilizes hidden layers to learn abstract feature representations.

4. **Level 4 - PatchTST:** Integrates deep neural layers with a multi-head attention mechanism to capture sequential structure and long-range dependencies in the data.

6.3 Experimental Setup

Following the methodology described in Section 4.6, the parameters are specified. After the preprocessing step, which resulted in the removal of nine tickers, the dataset consists of $n \in \{1, 2, \ldots, 92\}$ individual tickers. The chosen horizons for this study are $H = \{6, 12, 24, 48\}$, which corresponds to forecasting horizons of 30, 60, 120 and 240 minutes respectively. Therefore, the models prediction vector for each time step t and ticker n results in the dimension $\hat{\mathbf{y}}_{t,n} \in \mathbb{R}^4$.

6.4 Metrics

To provide a robust method for model evaluation, two sets of metrics are used. First, statistical methods to evaluate the point forecast accuracy of the models. Second, economic metrics are utilized to evaluate the models performance with a simulated trading strategy.

6.4.1 Statistical Metrics

These metrics evaluate the error between the predicted returns and the actual ground-truth values.

Mean Squared Error: The MSE penalizes large outliers quadratically.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{6.3}$$

Mean Absolute Error: The Mean Absolute Error (MAE) is a more robust measure against outliers because differences are not squared.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{6.4}$$

6.4.2 Economic Evaluation - Mean-Variance Analysis

To evaluate the economic performance, a Mean-Variance Optimization (MVO) is performed. This framework assesses the profitability of the model as a systematic investment strategy, providing a practical perspective in contrast to the statistical metrics. At every m-th timestep t, where $m = h \in H$ an optimal portfolio is constructed. This process takes two primary inputs: expected returns and a risk model.

Expected Returns: The models point predictions are used as the estimates for future returns. These are annualized based on the specific horizon h.

Risk Model: The covariance matrix is calculated using historical returns from the training set to avoid look-ahead bias. For this step, Ledoit-Wolf shrinkage is utilized to ensure a stable covariance matrix.

The ground-truth returns of the tickers are multiplied by the resulting weight vector of the optimal portfolio to determine the performance. This procedure is applied across the entire test set, over all horizons H, resulting in the annualized return and the annualized Sharpe ratio for each of the four horizons respectively. The annualized return represents the total return, the strategy would yield over a one-year period. The annualized Sharpe ratio measures the risk-adjusted performance. It indicates how much return is generated per unit of risk, which is defined as the standard deviation of the returns. While portfolio rebalancing costs are not considered, the results still provide a risk-adjusted view of the models.

6.5 Training Process with Hyperparameter Tuning

The models hyperparameters are selected through a hyperparameter tuning process. Due to the high computational demand of training all configurations across every complexity level C_k, the process was restricted to a subset of $N = 5$ tickers.

The training process is performed using a specific subset of the available hyperparameter configurations. After the training phase, each configuration is evaluated by generating predictions on the validation dataset. The evaluation is performed between the predicted values $\hat{\mathbf{y}}_{t,n}$ of the model and the ground-truth values $\mathbf{y}_{t,n}$ using the loss function MAE.

This process is performed for 10 iterations, where each configuration is randomly sampled from the available hyperparameter space. After the hyperparameter tuning process has finished, the configuration with the lowest validation loss is selected for the final model training on the combined training and validation sets.

The training was conducted on a workstation with an AMD Ryzen 7 3800X as CPU and an Nvidia 2060 Super as GPU.

6.5.1 Final Model Parameters

Linear Regression: The chosen baseline is an OLS linear regression, which was utilized without hyperparameter training.

RF: The search grid for the RF hyperparameters and the final selected parameters are presented in Appendix 7.2. For computational reasons due to the large dataset, the MSE was utilized as the split criterion instead of the MAE.

MLP: The architecture of the MLP was selected based on the parameter space provided in Appendix 7.2. It also represents the final configuration used for evaluation.

PatchTST: For the Transformer PatchTST, the parameter space and the resulting optimal parameters are shown in Appendix 7.2.

6.6 Quantitative Analysis

The model evaluation is based on both statistical metrics and economic performance. To assess whether the increasing levels of feature complexity C_k provide any predictive value, a "Constant Model" is introduced as a baseline. This baseline predicts a constant value for all timesteps in the training set and is optimized to minimize the MAE **without utilizing any** input features, for each horizon h respectively. Because of the mathematical properties of the MAE, the optimal constant predictor corresponds to the median of the target distribution. This model serves as a benchmark to determine if the more sophisticated models actually benefit from the different complexity levels or if a simple statistical average would yield similar results in the context of this study.

6.6.1 Machine Learning Performance Metrics

Evaluating all models on the test dataset in Table 6.3, it can be observed that all machine learning models perform relatively similar. However, nearly all of them either cannot outperform or perform only equally to the Constant Model. Complex models like PatchTST did not show any advantage over simpler models.

Furthermore, a more complex feature set, which resulted in an increasing dimension of the dataset, did not help to increase the signal-to-noise ratio. The fact that even the more complex models could not benefit from the computed features underlines the difficulties in this forecasting task.

6.6.2 Economic Evaluation Metrics

For the models economic evaluation, a mean-variance analysis was performed. The corresponding metrics are presented in Table 6.4.

Although relatively high return values can be observed, a comparison with the Constant Model reveals that these results are not caused by superior model performance, but by the models performing similar to the baseline. The strong performance of the Constant Model is primarily driven by a slightly positive median in the training distribution. As this upward trend continued throughout the testing period, the model effectively captured the market's positive drift, resulting in long-term profits without the need for any predictive features. Furthermore, these results do not account for transaction costs, which would significantly affect the outcome, especially for shorter horizons. For the horizon $h = 6$, the portfolio was rebalanced 2,929 times, whereas for the horizon $h = 48$, it was rebalanced 367 times over the testing period. These high turnover rates would add substantial transaction costs in an actual trading strategy.

Table 6.3: Statistical results: The MAE and MSE were computed on the unscaled predicted returns. The numbers are scaled to better visualize the results.

		Constant Model		Linear Regression		Random Forest		MLP		PatchTST	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Complexity	Horizon	(10^{-6})	(10^{-4})	(10^{-6})	(10^{-4})	(10^{-6})	(10^{-4})	(10^{-6})	(10^{-4})	(10^{-6})	(10^{-4})
C_{L10}	6	16.82	27.75	16.84	27.78	16.89	27.86	16.82	27.75	16.83	27.75
	12	35.30	40.89	35.36	40.95	35.63	41.19	35.31	40.90	35.35	40.93
	24	74.96	61.18	75.18	61.29	76.26	62.08	74.99	61.18	75.07	61.22
	48	156.57	90.95	157.36	91.22	161.26	93.39	156.74	90.98	157.06	91.04
C_{L20}	6	16.82	27.75	16.89	27.87	16.92	27.83	16.83	27.77	16.88	27.86
	12	35.30	40.89	35.56	41.17	35.71	41.13	35.37	40.94	35.53	41.11
	24	74.96	61.18	75.77	61.85	75.84	61.66	75.34	61.40	75.69	61.54
	48	156.57	90.95	159.11	92.28	158.35	91.78	157.53	91.36	158.47	91.63
C_{L30}	6	16.82	27.75	16.84	27.78	16.90	27.85	16.83	27.77	16.87	27.84
	12	35.30	40.89	35.38	40.97	35.65	41.14	35.34	40.94	35.39	40.96
	24	74.96	61.18	75.13	61.31	75.64	61.55	74.99	61.20	75.32	61.33
	48	156.57	90.95	156.81	91.11	157.40	91.32	156.42	90.88	157.96	91.42
$C_{L10+L20}$	6	16.82	27.75	16.91	27.88	16.91	27.87	16.84	27.77	16.85	27.79
	12	35.30	40.89	35.59	41.19	35.73	41.25	35.37	40.94	35.45	41.05
	24	74.96	61.18	76.01	61.94	76.08	61.93	75.17	61.27	75.43	61.52
	48	156.57	90.95	159.75	92.34	159.23	92.41	157.06	91.14	158.94	91.70
$C_{L10+L30}$	6	16.82	27.75	16.87	27.81	16.92	27.87	16.85	27.79	16.84	27.75
	12	35.30	40.89	35.49	41.04	35.74	41.26	35.38	40.94	35.36	40.91
	24	74.96	61.18	75.50	61.48	76.16	62.00	75.09	61.21	75.03	61.17
	48	156.57	90.95	157.86	91.43	159.06	92.41	156.75	91.02	157.40	91.26
$C_{L20+L30}$	6	16.82	27.75	16.89	27.85	16.92	27.83	16.83	27.76	16.88	27.80
	12	35.30	40.89	35.58	41.17	35.70	41.13	35.30	40.90	35.48	41.02
	24	74.96	61.18	75.96	61.92	75.78	61.61	74.96	61.17	75.40	61.34
	48	156.57	90.95	159.70	92.45	158.12	91.67	156.57	90.94	158.37	91.54
$C_{L10+L20+L30}$	6	16.82	27.75	16.91	27.88	16.93	27.86	16.82	27.75	16.91	27.86
	12	35.30	40.89	35.63	41.22	35.75	41.23	35.30	40.90	35.47	41.01
	24	74.96	61.18	76.17	62.02	76.00	61.83	74.96	61.18	75.56	61.43
	48	156.57	90.95	160.18	92.47	158.80	92.13	156.57	90.95	159.05	91.74

Table 6.4: Economic evaluation: The "Return" represents the annualized return and "Sharpe" the Sharpe ratio over the testing period respectively.

		Constant Model		Linear Regression		Random Forest		MLP		PatchTST	
		Return (%)	Sharpe	Return (%)	Sharpe	Return (%)	Sharpe	Return (%)	Sharpe	Return (%)	Sharpe
Complexity	Horizon										
C_{L10}	6	53.11	2.03	25.71	1.09	39.78	1.62	62.11	2.56	33.58	1.55
	12	58.45	2.15	40.35	1.77	33.45	1.27	57.12	2.21	40.37	1.81
	24	53.94	1.91	40.92	1.81	48.59	1.79	51.32	1.91	42.55	1.76
	48	51.42	1.77	38.43	1.73	54.55	1.97	49.07	1.81	43.81	1.79
C_{L20}	6	53.11	2.03	39.10	1.69	32.73	1.42	51.37	2.13	52.58	2.40
	12	58.45	2.15	40.84	1.70	32.18	1.34	61.96	2.58	45.54	2.04
	24	53.94	1.91	52.05	2.14	35.67	1.50	55.17	2.29	28.10	1.17
	48	51.42	1.77	49.97	2.03	58.68	2.30	43.73	1.79	22.76	0.86
C_{L30}	6	53.11	2.03	50.05	2.14	1.22	0.05	59.07	2.45	36.47	1.66
	12	58.45	2.15	45.00	1.86	-0.16	-0.01	57.28	2.24	41.07	1.74
	24	53.94	1.91	49.82	2.04	-0.53	-0.02	47.06	1.79	53.45	2.22
	48	51.42	1.77	44.71	1.60	30.85	1.15	46.60	1.68	33.33	1.39
$C_{L10+L20}$	6	53.11	2.03	44.64	1.99	44.09	1.88	59.19	2.68	42.29	1.86
	12	58.45	2.15	41.84	1.84	40.13	1.66	67.91	2.89	49.00	2.16
	24	53.94	1.91	44.10	1.96	54.45	2.19	61.25	2.62	49.00	2.09
	48	51.42	1.77	35.40	1.63	63.60	2.45	54.99	2.28	41.69	1.80
$C_{L10+L30}$	6	53.11	2.03	35.44	1.55	35.27	1.45	57.12	2.39	45.10	2.07
	12	58.45	2.15	41.98	1.90	14.26	0.55	48.47	1.97	46.68	2.04
	24	53.94	1.91	39.96	1.75	25.54	0.97	40.87	1.56	43.37	1.74
	48	51.42	1.77	34.37	1.52	21.86	0.78	37.73	1.33	37.01	1.59
$C_{L20+L30}$	6	53.11	2.03	51.69	2.23	21.04	0.91	53.20	2.04	41.00	1.81
	12	58.45	2.15	52.34	2.19	8.86	0.37	58.63	2.15	26.02	1.08
	24	53.94	1.91	56.43	2.40	29.76	1.23	53.78	1.94	33.15	1.29
	48	51.42	1.77	43.27	1.84	31.34	1.20	51.03	1.76	42.70	1.67
$C_{L10+L20+L30}$	6	53.11	2.03	47.28	2.11	29.76	1.26	53.11	2.03	50.36	2.37
	12	58.45	2.15	50.32	2.21	33.24	1.34	58.45	2.15	60.90	2.60
	24	53.94	1.91	38.05	1.73	43.59	1.77	54.39	1.91	51.61	2.21
	48	51.42	1.77	42.69	1.96	47.66	1.80	51.88	1.77	47.96	2.04

6.7 Qualitative Analysis

To understand the discrepancy between complex models and their limited predictive power, this section examines the underlying possible causes. The analysis focuses on four key areas:

- Dataset shifts
- Training dynamics
- Model predictions
- Impact of feature complexity and EMH

Dataset Discrepancy and Non-Stationarity

A primary challenge in financial forecasting is the non-stationarity of the data. Comparing the two main features (volume and return), from which most other features are calculated on, across the training, validation and testing sets, reveals a distribution shift (see Table 6.5).

Table 6.5: Descriptive statistics of scaled features (with robust scaling method) across datasets for AAPL, similar shifts can be observed for other tickers.

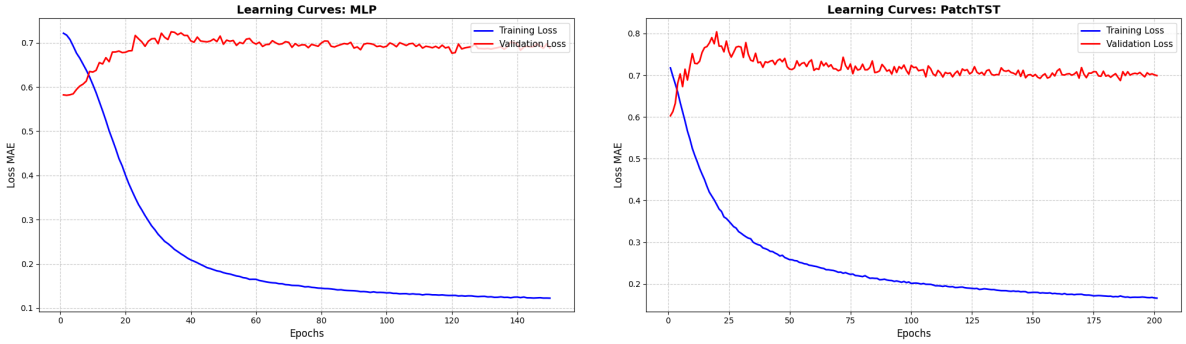
Feature	Dataset	Mean	Std	Median
LOGRET_1	Training	0.01	1.09	0.00
	Validation	0.01	1.34	0.00
	Test	0.01	0.91	0.02
LOG_VOLUME	Training	0.05	0.73	-0.00
	Validation	-0.39	0.57	-0.45
	Test	-0.78	0.55	-0.87

The discrepancy in performance can be partly explained by the shift in the log-volume feature. The median shifts from 0.00 in the training set to -0.87 in the test set. This suggests that the market environment during the testing period differed fundamentally from the training phase. Additionally, the standard deviation of 5-minute log-returns (LOGRET_1) increases from 1.09 to 1.34 between training and validation. In finance, this increase in volatility represents a shift in risk regimes. Such non-stationary behavior can explain why patterns learned during training fail to generalize to the validation set.

Training Dynamics and Overfitting

To further examine the model quality, the learning curves of the deep learning models (trained without early stopping on Complexity $C_{L10+L20+L30}$) are presented in Figure 6.10.

For the deep learning architectures (MLP and PatchTST), a rapid divergence between training and validation loss is observed. While the models are capable of reducing the training loss nearly to zero, the validation error remains stagnant or increases slightly. This indicates a lack of a learnable, generalizable signal.



(a) Learning curve for MLP

(b) Learning curve for PatchTST

Figure 6.10: Training curves

Predictabilty of Models

To gain a clearer understanding, the model predictions for PatchTST is visualized against the ground truth (see Figure 6.11). A key observation is that all models exhibit a significantly lower standard deviation in their predictions compared to the actual target variables (summarized in Table 6.6).

Mathematically, this behavior is a consequence of the chosen loss functions. When a model minimizes the MSE in an environment with a low signal-to-noise ratio, the optimal strategy, without any learnable patterns, is to predict the expected value:

$$\hat{y} = E[y|x] \approx \bar{y}$$

Financial time series exhibit a low signal-to-noise ratio. If the data does not represent enough predictive information, the model attempts to use the average to minimize the effect of outliers. The model defaults to the "safest" statistical estimate, the mean

Table 6.6: Comparison of statistical properties: Ground truth vs. model predictions of all models (Horizon $h = 6$ and complexity $C_k = C_{L10+L20+L30}$)

Model	**Mean** (10^{-4})	**Median** (10^{-4})	**Vol.** (σ) (10^{-4})
Ground Truth (y)	**1.0086**	**1.4515**	**41.0195**
Constant Model	0.8631	0.8631	0.0000
Linear Regression	0.4108	0.2734	2.9257
Random Forest	-0.0478	-0.0575	2.9138
MLP	0.7690	0.5573	0.3127
PatchTST	0.4323	0.5644	2.7786

of the distribution, which is approximately zero. For models optimized via MAE, the equivalent "safe" strategy is predicting the median. Since log-returns are approximately symmetrically distributed around zero (median ≈ mean ≈ 0), both strategies result in mean-like predictions that fail to capture profitable price movements.

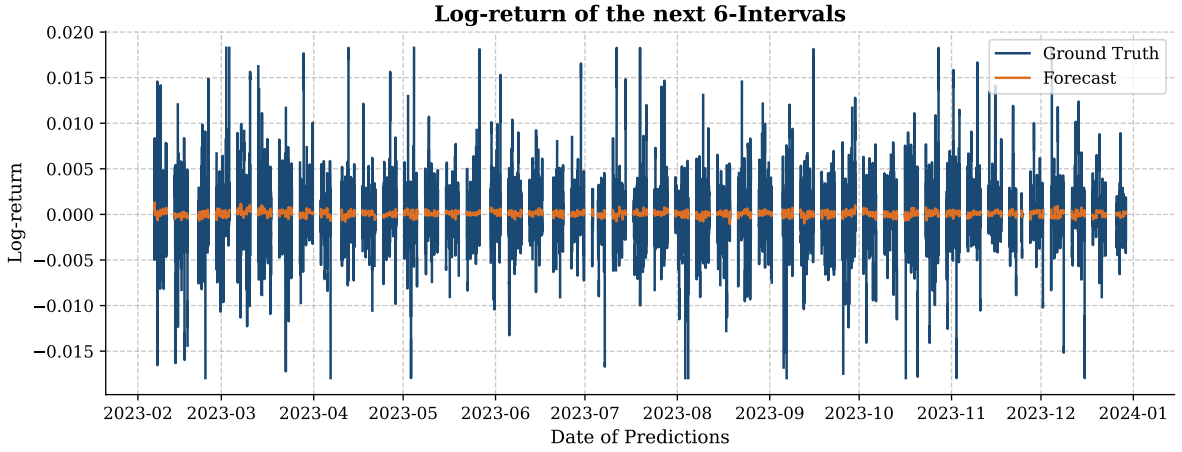


Figure 6.11: Comparison of predicted log-returns compared to ground truth for PatchTST (Horizon $h = 6$ and complexity $C_k = C_{L10+L20+L30}$).

Feature Complexity and Market Efficiency

The introduction of increasingly complex features (C_{L10} to $C_{L10+L20+L30}$) did not lead to a measurable impact on performance. This suggests that the "Microstructure Noise" at the 5-minute resolution exhibits the true signal. These findings align with the EMH, suggesting that at this frequency, price movements closely follow a random walk. This, makes it nearly impossible for models to extract generalizable patterns besides learning the long term trend using only historical OHLCV data.

7 Conclusion

7.1 Summary of Findings

This study demonstrates that predicting the short-term returns of individual tickers remains a very challenging task. It shows that models ranging from simple linear regression to state-of-the-art Transformers like PatchTST struggle to extract predictive information from technical indicators at a 5-minute resolution.

Instead of learning generalizable market dynamics, complex models exhibited immediate overfitting, while simpler models defaulted to a mean-prediction strategy. These results suggest that at high frequencies, the signal-to-noise ratio is too low for standard supervised learning on price data alone.

7.2 Future Research

This study focused on high-frequency intraday data but it is noteworthy that literature on daily returns often shows more promising results, as microstructure noise is less present [3].

Future research could expand upon these findings in several directions:

- **Alternative Data:** Including sentiment analysis or news feeds to provide non-price information.
- **Order Book Depth:** Using Limit Order Book (LOB) data instead of just OHLCV data to capture liquidity patterns.
- **Alternative Loss Functions:** Exploring "Directional Accuracy" loss functions instead of MSE/MAE.

Appendix

Appendix A: Detailed Feature Specification

Table 1: Overview of all features used in this thesis, categorized by complexity level. Feature names are listed exactly as they appear in `feature_config.json`. Indicator parameters can be retrieved from `feature_config.json`, and implementation details can be found in `technical_indicator.py`.

Name	**Cplx. Level**	**Category**	**Description**
`SMA`	10	Trend	The Simple Moving Average (SMA) calculates the arithmetic mean of log returns over the last intervals. It is used to smooth out price data and identify the underlying market trend. While the SMA over the last 20 intervals captures short-term trends within the 5-minute resolution, an SMA over the last 200 intervals serves as an indicators for medium- and long-term momentum.
`LOGRET`	10	Statistical/ Temporal	Logarithmic returns are calculated as the natural log difference of successive prices. Different prices in the horizons capture the dynamics of the returns respectively.
`LOG_VOLUME`	10	Volume	Applying a logarithmic transformation to trading volume reduces the skewness of the data, as volume often exhibits extreme spikes. It serves as an indicator of market activity and liquidity. An increase in log-volume validates the significance of a price movement, helping the model distinguish between meaningful market entries and random noise.
`EMA`	20	Trend	The Exponential Moving Average (EMA) applies more weight to recent data points. Compared to the SMA, the EMA reacts more quickly to abrupt price changes.
`Kurtosis`	20	Statistical/ Temporal	Kurtosis measures the tailedness of the return distribution over a set rolling interval window. In financial analysis, high kurtosis indicates a higher probability of extreme events. This feature can inform the model whether the market is in a stable state or if the risk of outliers is currently increased.

Table 1 – continued from previous page

Name	Cplx.Level	Category	Description
OBV	20	Volume	On-Balance Volume is a cumulative momentum indicator that relates volume to price change. It adds volume of the current interval when the current close price is higher than the previous close price otherwise it is subtracted.
DAY_OF_WEEK	20	Statistical/ Temporal	This categorical feature identifies the specific trading day (Monday-Friday) numericaly. It allows the model to potentially learn temporal patterns (lower returns on specific days, volatility patterns occurring at the beginning or end of the trading week).
MINUTE_OF_DAY	20	Statistical/ Temporal	This feature encodes the intra-day time component numericaly. In high-frequency 5-minute data, this can be helpful for learning time-of-day specific behaviors, such as high-volatility patterns during market open and close.
DAILY_CLOSE	20	Trend	The daily close provides a more stable level of close price. It represents the last interval's close price of the previous trading day. The value could influence the opening sentiment of the current intervals opening day.
VOL_SPIKE	20	Volume	A volume spike is calculated as the ratio of current volume to its moving average over n-periods.
VOL_RATIO	20	Volume	The volume ratio compares a short-term (fast) moving average of volume to a long-term (slow) moving average. It acts as a normalized momentum oscillator for market activity. It is more stable to noise than volume spike indicator.
VOLATILITY_RETURN	20	Volatility	This feature calculates the rolling standard deviation of log returns over various horizons. In a 5-minute context, it can help to understand the current risk environment.

Table 1 – continued from previous page

Name	Cplx.	Level	Category	Description
HL_RANGE	20		Volatility	The high-low range measures the maximum price spread (high price minus low price of each interval) over a specific window. It captures the total range covered by the price.
OC_RANGE	20		Volatility	The open-close range measures the absolute distance between the opening- and closing prices of each interval
TURNOVER	20		Volume	Turnover represents the total monetary value of shares traded during each interval.
RSI	30		Momentum	The Relative Strength Index (RSI) is a bounded oscillator that measures the speed and change of price movements.
ROC	30		Momentum	The price Rate of Change (ROC) is a momentum oscillator that measures the percentage change between the current price and the price n-periods ago. It quantifies the velocity of the trend. Positive ROC values indicate an accelerating uptrend, while negative values signal a strengthening downtrend
ADX	30		Trend	The Average Directional Index (ADX) quantifies trend strength without regard to direction.
VI	30		Trend	The Vortex Indicator (VI) consists of two lines (VI+ and VI-) that capture positive and negative trend movement. It is designed to identify the start of a new trend and its direction by measuring the distance between price extremes.
DMI	30		Trend	The Directional Movement Index (DMI) consists of DI+ and DI- which help to identify the direction of the trend. When +DI is above -DI, the upward pressure dominates.

Table 1 – continued from previous page

Name	Cplx.	Level	Category	Description
CCI	30		Momentum	The Commodity Channel Index (CCI) measures the current price relative to an average price over a given period. High values indicate that prices are unusually high compared to the average, while low values indicate they are unusually low.
MI	30		Volatility	The Mass Index (MI) examines the range between high and low prices over time. It is used to predict trend reversals by analyzing the movement of the price range.
STOCH	30		Momentum	The Stochastic Oscillator (STOCH) presents the location of the close price relative to the high-low price range over a specific number of periods. It follows the speed or momentum of price.
%BBANDS	30		Volatility	Percent Bandwidth (%B) quantifies the price relationship to the Bollinger Bands. A value of 1.0 means the price is at the upper band, while 0.0 means it is at the lower band.
MACD	30		Trend	The Moving Average Convergence Divergence (MACD) is a trend-following momentum indicator that shows the relationship between two moving averages of the close price. It consists of the MACD line (difference between 12 and 26-period EMAs close price) and a signal line (9-period EMA of the MACD line). Crossovers between these lines help the model identify shifts in trend direction.
PVO	30		Volume	The Percentage Volume Oscillator (PVO) is the volume-based equivalent of the MACD. It measures the difference between a fast and a slow EMA of volume. It allows the model to identify whether volume is increasing or decreasing relative to the long-term average.
NATR	30		Volatility	The Normalized Average True Range (NATR) is a measure of volatility that is normalized by the closing price.

Table 1 – continued from previous page

Name	Cplx.	Level	Category	Description
`MFI`	30		Volume	The Money Flow Index (MFI) is a technical oscillator that uses both price and volume to measure buying and selling pressure.

Appendix B: Setup Configuration (config.json)

Listing 1: Example configuration file for the pipeline setup.

```
{
    "data": {
        "concatenated_csv": "data/concatenated/",
        "number_of_tickers": 3,
        "start_date": "2015-01-01",
        "end_date": "2023-12-31",
        "use_cached_processed_data": false,
        "processed_csv": "data/processed/"
    },
    "model": {
        "modelType": "MLP"
    },
    "features": {
        "featureConfigPath":
   "financial_TSF/features/feature_config.json",
        "complexity_levels": [
            10,
            20,
            30
        ]
    },
    "pipeline": {
        "steps": [
            "DataLoadingStep",
            "FeatureStep",
            "SplittingStep",
            "NormalizingStep",
            "TrainingStep",
            "EvaluationStep",
            "SavingStep"
        ]
    },
    "results": {
        "resultsPath": "results/"
    },
    "settings": {
        "forecast_horizon": [
            48
        ]
    }
```

```
}
```

Appendix C: Pipeline Class

Listing 2: Implementation of pipeline class.

```
class Pipeline:
    def __init__(self, config_path: str = "financial_TSF/config.json"):
        with open(config_path, "r") as f:
            self.config: AppConfig = AppConfig.from_dict(json.load(f))
        self.model = MODEL_REGISTRY[self.config.model.model_type](self.config)
        self.steps: list[PipelineStep] = [
            STEP_REGISTRY[step_name](self.config) for step_name in self.config.pipeline.steps]

    def run(self, step_result: StepResult):
        step_result.model_instance = self.model
        for step in self.steps:
            print("-------------------------------------------------")
            print(f"Executing step: {step.__class__.__name__}")
            step_result = step.execute(step_result)
        return step_result
```

Appendix D: Base Model Interface

Listing 3: Abstract base class for all model implementations.

```
class BaseModel(ABC):

    @abstractmethod
    def fit(self, step_result: StepResult) -> Any:
        pass
    @abstractmethod
    def predict(self, step_result: StepResult) -> tuple:
        pass

    def tune_hyperparameters(self, step_result: StepResult) -> dict | None:
        pass
```

```
12
13      def save_model(self, step_result: StepResult) -> Any:
14          pass
15
16      def load_model(self, model_path: Path) -> Any:
17          pass
```

Appendix E: Base Model Interface

Listing 4: Abstract base class for each pipeline step.

```
1  class PipelineStep(ABC):
2      @abstractmethod
3      def execute(self, step_result: StepResult) -> Any:
4          pass
```

Appendix F: Feature Configuration (feature_config.json)

Listing 5: Feature Configuration - shortened for readability - complete file listed in code repository.

```
1  {
2    "technical_indicators": {
3      "SMA": {
4        "enabled": true,
5        "params": [
6          20,
7          50,
8          200
9        ],
10       "complexity": 10
11     },
12
13     ...
14
15     "OC_RANGE": {
16       "enabled": true,
17       "params": [
18         5,
19         10,
```

```
        20,
        40
      ],
      "complexity": 20
    },
    "TURNOVER": {
      "enabled": true,
      "complexity": 20
    },
    "LOG_VOLUME": {
      "enabled": true,
      "complexity": 10
    }
  }
}
```

Appendix G: RF Parameter Space

Table 2: Hyperparameter tuning space and best parameters for RF models across all horizons.

Parameter	**Tuning Space**	C_{L10}	$C_{L10+L20}$	$C_{L10+L20+L30}$	$C_{L10+L30}$	C_{L20}	$C_{L20+L30}$	C_{L30}
`Number of estimators`	[100, 200, 300, 500]	500	500	500	500	500	500	500
`Maximum tree depth`	[10, 15, 20, 30, None]	15	15	15	15	15	15	15
`Minimum samples leaf`	[10, 20, 40, 60, 100, 200, 500]	100	100	100	100	100	100	100
`Minimum samples split`	[2, 5, 10, 20, 50, 100]	20	20	20	20	20	20	20
`Maximum features`	[”sqrt”, ”log2”, 0.3, 0.5, None]	log2	log2	log2	log2	log2	log2	log2
`Split criterion`	no search applied	MSE	MSE	MSE	MSE	MSE	MSE	MSE
`Loss function`	no search applied	MSE	MSE	MSE	MSE	MSE	MSE	MSE

Appendix H: MLP Parameter Space

Table 3: Hyperparameter tuning space and best parameters for MLP models across all horizons.

Parameter	**Tuning Space**	C_{L10}	$C_{L10+L20}$	$C_{L10+L20+L30}$	$C_{L10+L30}$	C_{L20}	$C_{L20+L30}$	C_{L30}
`hidden layers`	[512, 256, 128, 64], [256, 256, 128, 64], [256, 256, 128], [256, 128, 64], [128, 128]	[128,64,32]	[128,128]	[128,64,32]	[256,256,128]	[128,64,32]	[256,256, 128,64]	[512, 256, 128, 64]
`dropout`	[0.1, 0.2, 0.3, 0.5, 0.6, 0.7, 0.8]	0.5	0.3	0.1	0.6	0.7	0.2	0.8
`learning rate`	[1E-3, 5E-4, 1E-4]	1.00E-04	5.00E-04	1.00E-04	5.00E-04	5.00E-04	1.00E-04	1.00E-04
`weight decay`	[0.0, 1E-5, 1E-4, 1E-3]	1.00E-05	1.00E-03	1.00E-03	1.00E-05	1.00E-05	1.00E-04	1.00E-04
`batch size`	[64, 128, 256, 512]	64	64	128	64	64	384	64
`patience`	no search applied	30	30	30	30	30	30	30
`loss function`	no search applied	MAE	MAE	MAE	MAE	MAE	MAE	MAE

Appendix I: PatchTST Parameter Space

Table 4: Hyperparameter tuning space and best parameters for PatchTST models across all horizons.

Parameter	**Tuning Space**	C_{L10}	$C_{L10+L20}$	$C_{L10+L20+L30}$	$C_{L10+L30}$	C_{L20}	$C_{L20+L30}$	C_{L30}
`Context length`	[32, 64, 128]	32	128	128	128	64	32	128
`Patch length`	[8, 16]	8	8	16	8	16	8	16
`Patch stride`	[4, 8]	8	8	4	8	8	4	8
`Model dimension`	[32, 64, 128]	32	32	32	32	64	32	64
`Number of attention heads`	[2, 4, 8]	4	2	2	2	2	2	2
`Number of hidden layers`	[2, 3, 4]	4	2	2	2	3	3	2
`FFN dimension`	[64, 128]	128	64	64	128	64	64	128
`Dropout`	[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]	0.8	0.4	0.5	0.2	0.4	0.8	0.3
`Head dropout`	[0.1, 0.2, 0.3, 0.4, 0.5, 0.6]	0.5	0.4	0.5	0.3	0.6	0.5	0.6
`Weight decay`	[1E-4, 5E-4, 1E-3, 1E-2]	1.00E-03	1.00E-03	1.00E-02	1.00E-03	1.00E-02	1.00E-04	5.00E-04
`Batch size`	[32, 64]	32	32	32	64	64	64	64
`Learning rate`	[1E-4, 3E-4, 1E-3]	3.00E-04	3.00E-04	3.00E-04	1.00E-04	1.00E-04	1.00E-04	3.00E-03

Table 4 – continued

`Patience`	no search applied	30	30	30	30	30	30	30
`Loss function`	no search applied	MAE	MAE	MAE	MAE	MAE	MAE	MAE

Listings

1 Example configuration file for the pipeline setup. 49
2 Implementation of pipeline class. 50
3 Abstract base class for all model implementations. 50
4 Abstract base class for each pipeline step. 51
5 Feature Configuration - shortened for readability - complete file listed in code repository. 51

Bibliography

[1] Ashish Vaswani et al. "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. 2017.

[2] Albert Gu and Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2024.

[3] Jongseon Kim et al. "A Comprehensive Survey of Deep Learning for Time Series Forecasting: Architectural Diversity and Open Challenges". In: *Artificial Intelligence Review* (2025).

[4] Ailing Zeng et al. *Are Transformers Effective for Time Series Forecasting?* 2022.

[5] Eugene F. Fama. "Efficient Capital Markets: A Review of Theory and Empirical Work". In: *The Journal of Finance* (1970).

[6] Burton G. Malkiel'. *A random walk down wall street*. W. W. NORTON & COMPANY, 2018.

[7] John J. Murphy. *Technical Analysis of the Financial Markets: A Comprehensive Guide to Trading Methods and Applications*. New York Institute of Finance, 1999.

[8] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. "Learning representations by back-propagating errors". In: *Nature* (1986).

[9] Leo Breiman. "Random Forests". In: *Machine Learning* (2001).

[10] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* (1997).

[11] Yuqi Nie et al. *A Time Series is Worth 64 Words: Long-term Forecasting with Transformers*. 2023.

[12] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020.

[13] Omer Berat Sezer, Mehmet Ugur Gudelek, and Ahmet Murat Ozbayoglu. "Financial time series forecasting with deep learning: A systematic literature review: 2005–2019". In: *Applied soft computing* (2020).

[14] Christopher Krauss, Xuan Anh Do, and Nicolas Huck. "Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500". In: *European Journal of Operational Research* (2017).

[15] Thomas Fischer and Christopher Krauss. "Deep learning with long short-term memory networks for financial market predictions". In: *European Journal of Operational Research* (2018).

[16] William F. Sharpe. "capital asset prices: a theory of market equilibrium under conditions of risk". In: *The Journal of Finance* (1964).

[17] Eugene F. Fama and Kenneth R. French. "Common risk factors in the returns on stocks and bonds". In: *Journal of Financial Economics* (1993).

[18] Mark M. Carhart. "On Persistence in Mutual Fund Performance". In: *The Journal of Finance* (1997).

[19] narasimhan jegadeesh narasimhan and sheridan titman sheridan. "Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency". In: *The Journal of Finance* (1993).

[20] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 2021.

[21] Ethem Alpaydın and Karen Lippert. *Maschinelles Lernen*. De Gruyter Oldenbourg, 2022.

[22] Martin J. Pring. *Technical Analysis Explained: The Successful Investor's Guide to Spotting Investment Trends and Turning Points*. McGraw-Hill, 2002.

[23] J. Welles Wilder. *New Concepts in Technical Trading Systems*. Trend Research, 1978.

[24] Xia Hu et al. "Model complexity of deep learning: a survey". In: *Knowledge and Information Systems* (2021).

Declaration on Oath

Hiermit versichere ich, dass ich die vorgelegte Bachelorarbeit selbstständig verfasst und noch nicht anderweitig zu Prüfungszwecken vorgelegt habe. Alle benutzten Quellen und Hilfsmittel sind angegeben, wörtliche und sinngemäße Zitate wurden als solche gekennzeichnet.

N. N., am April 8, 2026

Consent to Plagiarism Check

Hiermit willige ich ein, dass zum Zwecke der Überprüfung auf Plagiate meine vorgelegte Arbeit in digitaler Form an PlagScan (www.plagscan.com) übermittelt und diese vorübergehend (max. 5 Jahre) in der von PlagScan geführten Datenbank gespeichert wird sowie persönliche Daten, die Teil dieser Arbeit sind, dort hinterlegt werden.

Die Einwilligung ist freiwillig. Ohne diese Einwilligung kann unter Entfernung aller persönlichen Angaben und Wahrung der urheberrechtlichen Vorgaben die Plagiatsüberprüfung nicht verhindert werden. Die Einwilligung zur Speicherung und Verwendung der persönlichen Daten kann jederzeit durch Erklärung gegenüber der Fakultät widerrufen werden.

N. N., am April 8, 2026